

Versionado de Modelos y Gestión de Experimentos

Módulo 8 · Ingeniería de Modelos y LLMOps

Prompts como código, modelos pinned, MLflow, W&B, reproducibilidad hermetica y cadencia de experimentacion.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Versionado de Modelos y Gestión de Experimentos

En software tradicional, el versionado del código es la práctica más básica de ingeniería. En sistemas LLM, el "código" que determina el comportamiento del sistema incluye no solo el código Python sino también los prompts, los modelos base, los adaptadores fine-tuneados, los datos de evaluación, y la configuración del sistema. Sin versionado de todos estos componentes, es imposible reproducir el estado exacto del sistema en cualquier momento, entender qué cambio produjo una mejora o degradación, o hacer rollback a una versión anterior cuando algo falla.

La gestión de experimentos va más allá del versionado: es el proceso de diseñar, ejecutar, y analizar sistemáticamente los experimentos que mejoran el sistema. Un experimento LLM puede ser una variación del prompt, un cambio del modelo base, un ajuste de los parámetros de retrieval, o un nuevo fine-tuning. Sin un sistema que registre cada experimento con sus condiciones exactas y resultados reproducibles, el proceso de mejora es artesanal y no escalable.

Qué versionar en un sistema LLM

Prompts como código

Los prompts son el componente más frágil del sistema LLM y el que más frecuentemente cambia. Deben estar en control de versiones (Git) junto con el código. Cada cambio de prompt debe tener: un hash de commit que lo identifica unívocamente, una descripción del cambio y por qué se hizo, los resultados de evaluación antes y después del cambio sobre el golden dataset, y la fecha y responsable del cambio. Herramientas como LangSmith y Langfuse tienen registros de prompts con versionado integrado que simplifica este proceso.

Modelos base y adaptadores

Los modelos base cambian: los proveedores actualizan silenciosamente las versiones (gpt-4o-2024-11-20 → gpt-4o-2025-01-15), y estos cambios pueden afectar el comportamiento del sistema aunque el prompt sea idéntico. Siempre especifica la versión exacta del modelo en la configuración del sistema (nunca "gpt-4o-latest": usa "gpt-4o-2024-11-20"). Para modelos open source fine-tuneados, versiona los adaptadores LoRA junto con el hash del modelo base, el dataset de fine-tuning, los hiperparámetros, y las métricas de evaluación.

Golden datasets y métricas de evaluación

El golden dataset es el "test suite" del sistema LLM. Debe estar en control de versiones con el mismo rigor que el código de producción: cada actualización tiene un hash de commit, se documenta qué casos se añadieron o modificaron y por qué, y el sistema de CI/CD ejecuta automáticamente los tests sobre el golden dataset antes de cualquier despliegue.

Herramientas de experiment tracking para LLM

MLflow: el estándar de experiment tracking

MLflow es el framework de experiment tracking más adoptado en MLOps. Para LLM, se usa para registrar: el modelo o versión del API usada, los parámetros del prompt (temperatura, max_tokens, versión del prompt), las métricas de evaluación (faithfulness, answer relevancy, LLM-as-judge score), y los artefactos (el prompt exacto, el golden dataset, los outputs del modelo). MLflow tiene un Model Registry que permite promover modelos de staging a producción con aprobación, y trackear qué versión está en producción en cada momento.

Weights & Biases para LLM

W&B; es una plataforma de experiment tracking con mejor visualización que MLflow y mayor adopción en equipos de investigación. W&B; Tables permite comparar los outputs del modelo lado a lado para distintas versiones del prompt o del modelo. W&B; Sweeps automatiza la búsqueda de hiperparámetros. Para fine-tuning, W&B; loggea automáticamente las curvas de pérdida de entrenamiento y validación, permitiendo comparar runs con distintas configuraciones.

Hermetic reproducibility: el requisito de reproducibilidad

La reproducibilidad hermética requiere que todos los componentes del experimento estén pinned a versiones exactas: versión exacta del modelo (incluyendo la fecha del snapshot para modelos que cambian), versión del prompt en Git (hash de commit), versión del golden dataset (hash de commit), seed de muestreo si se usa temperatura > 0, y versión de todas las librerías (requirements.txt o pyproject.toml con versiones fijas). Sin estos, dos "ejecuciones del mismo experimento" pueden producir resultados distintos.

SECCIÓN 4 · EJEMPLOS REALES

- MLflow Model Registry para gestión de modelos fine-tuneados (empresa de software): cada versión del adaptador LoRA se registra en MLflow con su modelo base, hiperparámetros de entrenamiento, dataset de fine-tuning, y métricas de evaluación. El pipeline de CI/CD requiere que cualquier nuevo adaptador tenga métricas de evaluación >X en el golden dataset antes de promover a producción. El historial completo de todos los modelos fine-tuneados está en MLflow.
- Braintrust para A/B testing de prompts (e-commerce): el equipo usa Braintrust para comparar sistemáticamente variaciones de prompts. Cada variación se evalúa sobre el mismo golden dataset de 200 casos. El dashboard muestra side-by-side los scores de cada variación. La decisión de qué prompt desplegar se basa en datos, no en intuición. Reportan mejoras del 30%+ en accuracy con este proceso sistemático.
- Git + LangSmith prompt versioning (startup fintech): todos los prompts en Git bajo /prompts/. Cada PR que cambia un prompt incluye el resultado de la evaluación sobre el golden dataset en la descripción. LangSmith referencia el hash de Git en cada trace, permitiendo correlacionar cualquier interacción de producción con la versión exacta del prompt que la produjo.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar modelos sin versión pinned en producción. "gpt-4o-latest" en producción significa que el comportamiento del sistema puede cambiar silenciosamente cuando OpenAI actualiza el modelo. Siempre especifica la versión exacta (gpt-4o-2024-11-20) y evalúa explícitamente antes de migrar a una nueva versión.

Error 2: No trackear los experimentos fallidos. Los experimentos que no mejoran el sistema contienen información valiosa: saber qué no funciona evita repetir los mismos errores. Registra siempre todos los experimentos en MLflow o W&B; no solo los exitosos.

Error 3: Comparar experimentos con condiciones distintas. Si el experimento A se evaluó con golden dataset v1 y el experimento B con golden dataset v2, los resultados no son comparables. Todos los experimentos de una comparación deben usar exactamente el mismo golden dataset, el mismo evaluador, y las mismas condiciones.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

La estructura mínima de versionado para un sistema LLM: repositorio Git con: /prompts/ (todos los prompts versionados), /evaluation/ (golden dataset y scripts de evaluación), /config/ (configuración del sistema incluyendo versión exacta de modelos), /experiments/ (resultados de experimentos). Requirements.txt o pyproject.toml con todas las dependencias pinned. En CI/CD: ejecutar la evaluación sobre el golden dataset en cada PR que cambie prompts o configuración del sistema.

Para experiment tracking con MLflow: `mlflow.start_run()`, `mlflow.log_param("model_version", "gpt-4o-2024-11-20")`, `mlflow.log_param("prompt_hash", git_hash)`, `mlflow.log_metric("faithfulness", 0.89)`, `mlflow.log_metric("answer_relevancy", 0.92)`, `mlflow.log_artifact("outputs.json")`, `mlflow.end_run()`. El MLflow UI muestra todos los experimentos comparados en una tabla y permite visualizar las métricas en gráficos.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M8-T1 (Ciclo de vida LLM): el versionado y experiment tracking son los habilitadores del ciclo de mejora continua.
- M8-T7 (Observabilidad): los traces de producción alimentan el proceso de experimentación identificando qué mejorar.
- M6-T8 (Evaluación de prompts): el golden dataset y las métricas de evaluación son compartidos entre el proceso de experimentación y la evaluación de prompts.

SECCIÓN 8 · RESUMEN EJECUTIVO

El versionado de sistemas LLM cubre: prompts en Git (con hash de commit + descripción del cambio + métricas), modelos con versión exacta pinned (nunca "latest"), adaptadores fine-tuneados con su modelo base + hiperparámetros + dataset, y golden datasets. La reproducibilidad hermética requiere pin de todas las versiones: modelo, prompt, dataset, seed, librerías. Herramientas: MLflow (estándar, Model Registry, CI/CD integration), W&B; (mejor visualización, fine-tuning tracking), Braintrust (mejor para prompt A/B testing sistemático). La cadencia mensual de experimentación convierte el sistema LLM en uno

que mejora continuamente.